

MCT231: Basics of Mechatronics

Final Submission Report

1	Ahmed Mohamed Taha	24P0050
2	Nada Abdelhamid	2301102
3	Mohamed Samy Taha	24P0373
4	Rawaa Tarig Algadi	2101367
5	Farida Mohamed Saeed	24P0008



Table of Contents

1. Introduction.....	3
2. Work Breakdown Structure.....	4
3.System Flowchart	5
4. System Block Diagram.....	6
5. Product Design Specifications.....	6
6. Actuator Sizing.....	7
7. Software Designs.....	13
7.2. CAD Assembly Design.....	16
7.3 Wiring Diagrams.....	16
7.4. PCB Design.....	19
8. Arduino Programming.....	19
9. Contribution Table.....	21
10. Conclusion.....	22
11. References.....	23

Introduction:

Advances in mechatronic systems have enabled the development of interactive products that combine mechanical design, electronics, control systems, and software into a single integrated solution. Educational toys are an important application of mechatronics because they provide an engaging environment where children can learn through play while interacting with automated systems. By incorporating sensors, actuators, user interfaces, and feedback mechanisms, educational toys can improve children's cognitive development, problem-solving skills, and engagement. This project presents the design and implementation of a fully automated SpongeBob-themed educational toy that combines ball dispensing, lifting, shooting, sorting, and user interaction mechanisms within a closed-loop mechatronic system. The toy is designed to entertain children while teaching them color recognition through an interactive question-and-answer game. The system uses multiple sensors to detect the presence and type of ball, actuators to control ball movement throughout the toy, and a graphical user interface (GUI) that allows monitoring and control from a personal computer through Bluetooth communication.

The toy operates using a sequence of interconnected mechanisms. A dispensing mechanism releases a ball only when a sensor confirms its presence. The ball then enters a collection box where another sensor detects its arrival and initiates an educational activity. At this stage, a 16×2 I²C liquid crystal display (LCD) prompts the child with a question asking them to identify the color of the ball. The child responds using one of two push buttons corresponding to the possible ball colors. Once a response is selected, the lifting mechanism transports the ball upward toward the shooting mechanism. A DC motor controlled through an L298N motor driver and monitored by an optical encoder launches the ball using a PID-controlled shooting system. Finally, a sorting mechanism equipped with infrared sensors identifies the ball type and directs it to the appropriate output path using a servo-actuated gate.

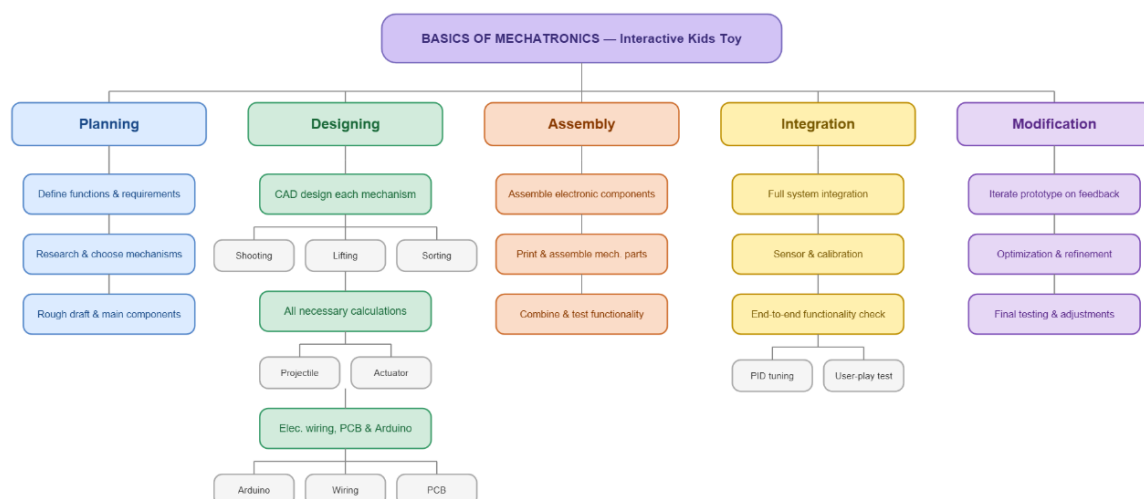
The system employs a closed-loop control strategy through the use of infrared sensors and an optical encoder that continuously provide feedback to the microcontrollers. This feedback enables reliable operation, accurate ball detection, controlled motion, and automated decision-making throughout the toy. Two Arduino Nano microcontrollers are used to distribute system tasks and improve modularity. The Master Nano manages

the user interface, Bluetooth communication, LCD display, scoring system, dispenser mechanism, and game logic, while the Slave Nano controls the lifting, shooting, and sorting mechanisms. Communication between the two microcontrollers is achieved through the I²C protocol, while wireless communication with the computer GUI is established using an HC-06 Bluetooth module.

To power the system, a rechargeable 12 V lithium battery pack serves as the primary energy source. A buck converter is used to regulate the voltage and provide a stable 5 V supply for the microcontrollers, sensors, servos, Bluetooth module, LCD display, and other low-voltage electronic components, while the DC motor and motor driver operate directly from the 12 V supply. This power distribution architecture ensures efficient operation and reliable performance across all subsystems.

The primary objective of this project is to demonstrate the integration of mechanical design, embedded systems, sensor feedback, motor control, wireless communication, and human-machine interaction into a single mechatronic product. Through the implementation of automated ball handling, PID-based motion control, interactive educational gameplay, and real-time feedback, the project provides a practical application of the concepts studied in mechatronics engineering while delivering an engaging learning experience for children.

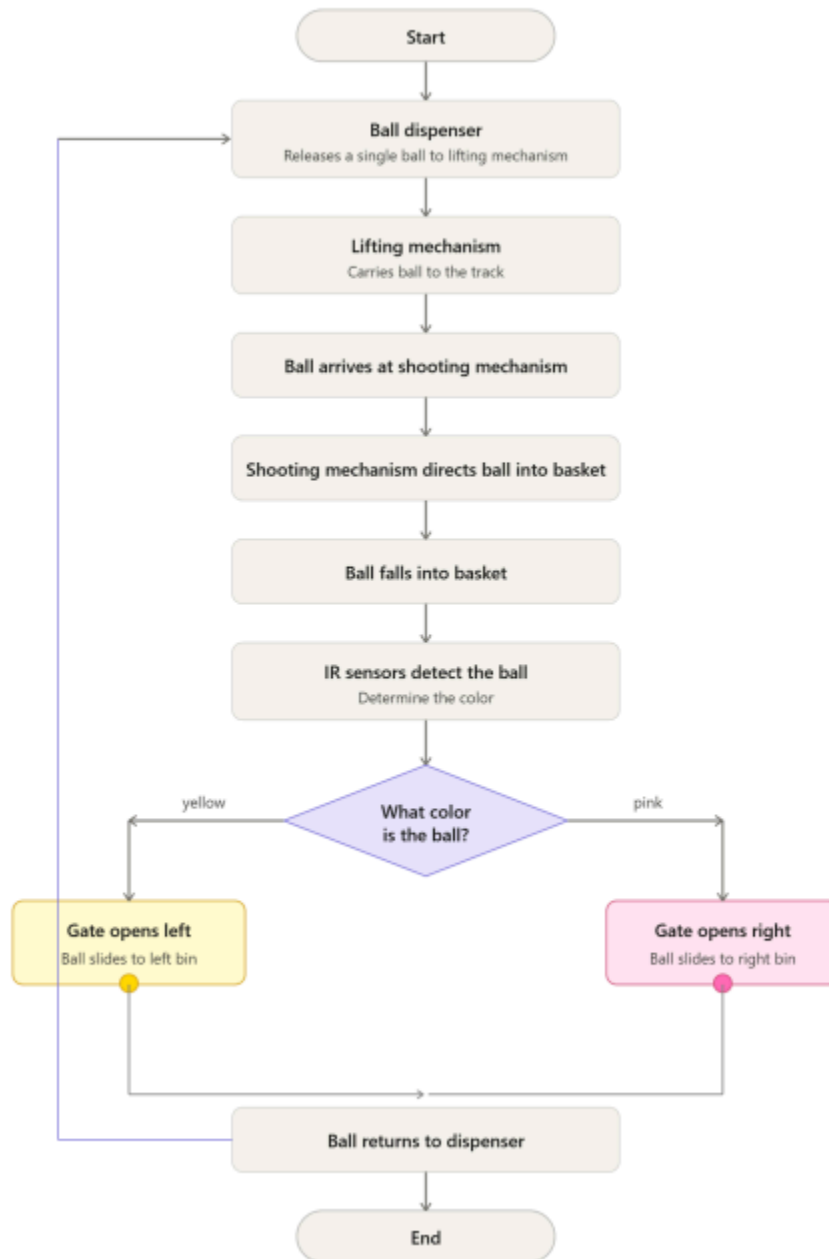
Work Breakdown Structure :



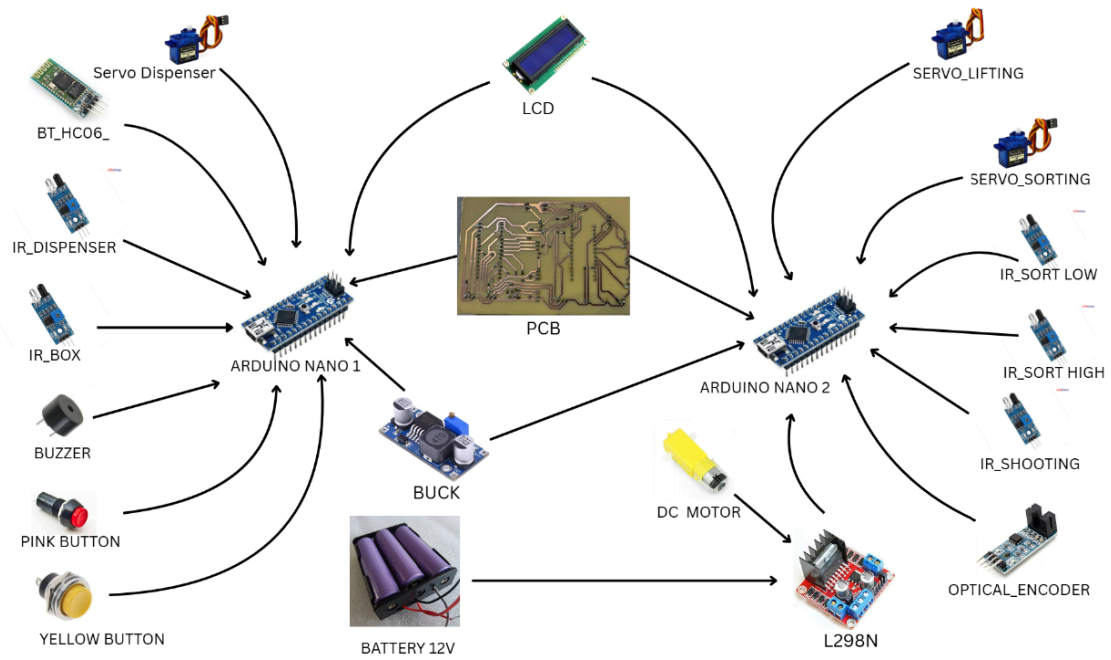
Gantt Chart:



System flowchart:



System block diagram:



Product design specifications:

1. Product Purpose & Overview

This Product Design Specification (PDS) defines all functional, physical, electronic, and safety requirements for the Automated Ball Sorting and Shooting System — an interactive mechatronic toy developed as part of the MCT231 Basics of Mechatronics major project at Ain Shams University.

The system automates the complete lifecycle of a ball from dispenser entry to sorted launch: detecting each ball via IR sensors, classifying it by color, routing it through a sorting gate, elevating it via a belt-driven lifting mechanism, and finally launching it toward a target basket using a rotating-arm shooting mechanism.

1.1 Project Vision

To design a safe, engaging, and durable interactive toy that demonstrates real-world closed-loop mechatronic control — merging mechanical precision, embedded electronics, and automated feedback in a compact desktop form factor.

1.2 Intended Users

Primary users are children (supervised play). Secondary users include engineering students and educators demonstrating mechatronic integration concepts.

2. Functional Requirements

2.1 Core Functions

ID	Function	Description	Performance Target
F1	Ball Detection	IR sensors detect ball presence at the sorting gate	Response < 50 ms
F2	Color Sorting	IR dual-sensor pair classifies ball (orange / gray) and actuates rotary gate	Sorting accuracy $\geq$ 95%
F3	Vertical Lifting	Belt-drive mechanism elevates ball to launch height	Height: target rail level

ID	Function	Description	Performance Target
F4	High-Velocity Launch	Rotating arm releases ball at calculated velocity toward basket	Range: 30–35 cm
F5	Ball Return	Ball returns to dispenser after landing for repeated cycles	< 2.0 s cycle time
F6	Feedback	LED/sound signals indicate successful hits	Immediate (< 200 ms)

2.2 Performance Specifications

Parameter	Value	Notes
Sorting gate precision	$\pm 5^\circ$	Rotary gate alignment with exit tracks
Total cycle time	$\leq 2.0 \text{ s / ball}$	Detection $\rightarrow$ launch
Launch landing variance	$< 5 \text{ cm}$	Shot repeatability
Required launch velocity	$\approx 2.7 \text{ m/s}$	At 35° launch angle
RMS launch velocity	$\approx 1.21 \text{ m/s}$	Continuous duty cycle
Lifting target velocity	0.5 m/s	Belt drive peak speed
Lifting peak torque	$4.84 \text{ N}\cdot\text{m}^2$	Trapezoidal velocity profile

Parameter	Value	Notes
Lifting RMS torque	$\approx 3.16 \text{ N}\cdot\text{m}^2$	1.0 s cycle

3. Physical & Dimensional Constraints

Constraint	Specification	Rationale
Footprint	$\leq 600 \text{ mm} \times 600 \text{ mm}$	Compact desktop use (per CAD assembly)
Total system weight	$< 1.5 \text{ kg}$	Portability
Ball radius	18 mm	PLA, ~15% infill, $\approx 7.3 \text{ g}$
Pulley radius	10.5 mm	Belt-drive lifting pulley

Constraint	Specification	Rationale
Arm length (shooting)	50 mm	Rotating arm projectile release
Belt incline angle	55°	Lift mechanism geometry
Primary material	PLA / TPU (3D printed)	Rapid prototyping, low cost, child-safe

4. Electronic & Control Specifications

4.1 Microcontroller

Arduino Uno / Nano used as the central controller. It receives digital signals from IR sensors, outputs PWM signals to servo motors, and drives the DC motor via an H-bridge module.

4.2 Sensors

Sensor	Type	Function
Ball Detection Sensor	IR Obstacle Avoidance (digital)	Detects ball arrival at sorting station
Color Sensor (pair)	Dual IR proximity sensors (Low/High pins)	Distinguishes orange vs. gray balls for routing

4.3 Actuators

Actuator	Type	Output	Role
Sorting Servo	SG90 Micro Servo	45° rotation	Rotary gate redirects ball by color
Lifting Servo/Stepper	High-torque servo or stepper	0–90° / steps	Belt-drive pulley for vertical elevation
Shooting Motor	N20 DC Gear Motor	~400 rpm / ≈ 2.7 m/s	Drives rotating arm for ball launch

4.4 Motor Driver

L298N or L293D dual H-bridge module. Input: PWM signals from Arduino. Output: variable voltage to DC motor.

Separate 12 V supply for motor power; 5 V logic from Arduino.

4.5 Power Supply

Dual-rail supply: 5 V for logic/servos (USB or regulated supply), 12 V for DC shooting motor (external battery pack or adapter).

Total estimated current draw < 2 A under peak load.

5. Actuator Sizing Summary

5.1 Lifting Mechanism

Parameter	Value	Formula / Note
Total payload mass	40 g	Belt (30 g) + holder + ball (10 g)
Linear acceleration	2.5 m/s ²	$v/t_a = 0.5/0.2$
Angular acceleration	238.1 rad/s ²	a / r_p

Parameter	Value	Formula / Note
Peak torque (T _{peak})	4.84 N·m ²	T _{load} + T _{friction} + T _{acc}
RMS torque (T _{rms})	≈ 3.16 N·m ²	Trapezoidal profile, 1.0 s cycle

5.2 Shooting Mechanism (Rotating Arm)

Parameter	Value	Formula / Note
Required release velocity	≈ 2.06 m/s	Projectile motion to 30 cm range
Angular velocity	41.2 rad/s (≈ 394 rpm)	v / L
Peak torque (τ _{peak})	0.0157 N·m	τ _{inertia} + τ _{gravity}

Parameter	Value	Formula / Note
RMS torque (τ_{rms})	$\approx 0.0070 \text{ N}\cdot\text{m}$	0.2 s accel / 1.0 s cycle

5.3 Sorting Mechanism (Rotary Gate)

Parameter	Value	Formula / Note
Rotation angle	45° (0.785 rad)	One sorting sweep
Angular acceleration	39.25 rad/s^2	$2\theta / t^2$
Peak torque (τ_{peak})	$0.00365 \text{ N}\cdot\text{m}$	$\tau_{inertia} + \tau_{gravity}$
RMS torque (τ_{rms})	$\approx 0.00165 \text{ N}\cdot\text{m}$	0.2 s motion / 1.0 s cycle

6. Projectile Motion Calculations

Parameter	Value	Note
Horizontal target distance (d)	30.76 cm	Design target range
Vertical drop to basket (h)	12 cm	Height differential
Launch angle (θ)	35°	Optimized for stable curved path
Required initial velocity (v)	≈ 2.7 m/s	From projectile equation
RMS velocity (V_{rms})	≈ 1.21 m/s	Continuous thermal load (1.0 s duty)

Governing equation: $v = \sqrt{[(g \cdot d^2) / (2 \cdot \cos^2(\theta) \cdot (d \cdot \tan(\theta) - h))]}$

7. Safety & Quality Standards

Category	Requirement
Rotating parts	High-speed rotating components (shooting arm, belt pulleys) must be partially shielded to prevent accidental contact
Structural stability	Base must be weighted or suctioned to prevent tipping during high-acceleration launch phases
Projectile safety	Low-velocity launch (< 3 m/s); soft PLA balls only; no sharp edges
Electrical safety	All high-voltage connections enclosed; USB/DC input only; no exposed mains wiring

Category	Requirement
Budget constraint	Prototype component cost $\leq$ 5,000 EGP
Material safety	Primary structure 3D-printed in PLA/TPU; child-safe, non-toxic

8. System Integration

System integration is the process of combining all independently designed subsystems — mechanical, electronic, and software — into a single cohesive and functional unit.

For this project, integration bridges four distinct domains: the sorting mechanism, the lifting mechanism, the shooting mechanism, and the Arduino-based control layer.

8.1 Integration Architecture

The system follows a sense–process–actuate closed-loop architecture coordinated by a central Arduino Uno:

Layer	Component	Interface	Signal Type
Sense	IR Sensor (detection)	Digital pin (Arduino)	Digital HIGH/LOW
Sense	IR Sensor pair (color)	Digital pins 2 & 4	Dual-bit color code
Process	Arduino Uno	Central MCU	Logic / PWM generation

Layer	Component	Interface	Signal Type
Actuate	SG90 Servo (sorting)	PWM pin 0	0°–55° position
Actuate	Servo (lifting)	PWM pin 9	90°–100° sweep
Actuate	N20 Motor (shooting)	L298N via pins 12/13/6	PWM speed + direction

8.2 Subsystem Sequencing & Timing

The Arduino orchestrates all subsystems in a strict timed sequence within a 2.0-second cycle budget:

Step	Subsystem	Action	Duration	Trigger Condition
1	Sorting (IR + Servo)	Ball detected → color classified → gate opens to correct lane	0.2 s	IR sensor goes HIGH

Step	Subsystem	Action	Duration	Trigger Condition
2	Sorting (Servo reset)	Gate returns to neutral position	0.1 s	After 1000 ms delay
3	Lifting (Servo/motor)	Belt drive elevates ball to launch rail height	0.4 s	After sorting complete
4	Shooting (N20 motor)	Arm accelerates → releases ball at 2.7 m/s toward basket	0.2 s	Ball present at launch cup
5	Motor idle / reset	Motor decelerates; system ready for next ball	0.1 s	After shoot delay
Total	—	Full ball cycle	≤ 2.0 s	—

8.3 Software Integration (Arduino Control Logic)

The unified Arduino sketch integrates all three subsystem programs (lifting.ino, dc.ino, sortingfinal.ino) into a single sequential control loop:

```
// Integrated Control Flow (pseudo-code)
void loop() {
  int low = digitalRead(sensorLow); // Color IR low bit
  int high = digitalRead(sensorHigh); // Color IR high bit
  if (low==0 && high==1) { gate.write(0); delay(1000);
gate.write(0); } // Small (orange) ball
  else if (high==0) { gate.write(55); delay(1000); gate.write(0); } //
Large (gray) ball
  beltServo.write(100); delay(1000); beltServo.write(90); // Lift
  // DC motor shoots → stops (L298N H-bridge logic)
}
```

8.4 Electrical Integration

All subsystems share a common GND. The L298N motor driver receives a dedicated 12 V supply for the shooting motor while drawing 5 V logic from the Arduino.

Servo power is supplied via the Arduino 5 V rail (current-limited; external 5 V BEC recommended under full load).

Signal routing is managed through a custom PCB designed in Fritzing/EasyEDA.

Component	Power Rail	Signal Pin(s)	Driver/Module
IR Sensors (×2)	5 V / GND	Pins 2, 4	Direct to Arduino
SG90 Sorting Servo	5 V / GND	Pin 0 (PWM)	Direct to Arduino
Belt Drive Servo	5 V / GND	Pin 9 (PWM)	Direct to Arduino
N20 Shooting Motor	12 V (motor) / 5 V (logic)	Pins 12, 13, 6	L298N H-bridge

8.5 Mechanical Integration

All subsystems are mounted on a shared 600 mm × 600 mm 3D-printed base plate.

The ball track connects the sorting gate exit directly to the lifting mechanism input rail, which feeds the rotating arm cup.

Alignment tolerances are maintained within ± 2 mm across the track junctions to ensure smooth ball transition without jamming.

Interface Point	Connected Subsystems	Tolerance / Note
Dispenser → Sorting gate	Ball dispenser ↔ IR sensor + rotary gate	Ball channel: ± 1 mm alignment
Sorting gate → Lift input	Rotary gate exit ↔ belt drive bottom	Gravity-fed track; slope $\geq 15^\circ$
Lift output → Shoot cup	Belt drive top rail ↔ rotating arm cup	Horizontal rail; ± 2 mm height
Shoot cup → Basket	Rotating arm ↔ target basket	Projectile path; 30–35 cm free air

8.6 Integration Testing Plan

Each subsystem must pass individual unit tests before full integration testing proceeds:

#	Test	Pass Criterion	Method
T1	IR sensor detection	Ball detected within 50 ms	Oscilloscope / Serial Monitor
T2	Color classification accuracy	$\geq 19/20$ balls correctly sorted	Manual run of 20 balls
T3	Sorting gate rotation	Gate reaches $\pm 5^\circ$ of target angle	Protractor + servo feedback
T4	Lifting mechanism	Ball reaches top rail within 0.4 s	Stopwatch timing
T5	Shooting accuracy	Ball lands within 5 cm of target	10-shot grouping test

#	Test	Pass Criterion	Method
T6	Full cycle integration	Complete cycle ≤ 2.0 s; no jams	End-to-end timed run $\times 10$
T7	Repeated operation	50 consecutive cycles without failure	Endurance run

9. Bill of Materials (Summary)

#	Component	Qty	Subsystem	Notes
1	Arduino Uno	1	Control	Central MCU
2	SG90 Micro Servo	2	Sorting + Lifting	PWM-controlled positioning

#	Component	Qty	Subsystem	Notes
3	N20 DC Gear Motor	1	Shooting	~400 rpm at rated voltage
4	L298N H-Bridge Module	1	Motor Driver	12 V motor / 5 V logic
5	IR Obstacle Sensor	2	Sensing	Ball detection & color classification
6	PLA Filament	~200 g	Structure	Frame, arm, gate, pulleys, balls
7	TPU Filament	~50 g	Belt	Flexible belt drive

#	Component	Qty	Subsystem	Notes
8	PCB (custom)	1	Electronics	Designed in Fritzing / EasyEDA
9	Power supply (5 V / 12 V)	1	Power	Dual-rail DC adapter or battery
10	Jumper wires, headers	—	Wiring	Interconnects

Estimated total prototype cost: < 5,000 EGP

Actuator sizing & projectile calculations:

Subsystem	Actuator type	Key output	Drive
Lifting mechanism	High-torque servo / stepper	0.5 m/s belt speed	PWM pin 9
Shooting mechanism	N20 DC gear motor	~400 rpm / 2.7 m/s	L298N + pins 12/13/6

Subsystem	Actuator type	Key output	Drive
Sorting mechanism	SG90 micro servo	45° gate sweep	PWM pin 0

2. Lifting Mechanism

Belt-drive servo/stepper motor

2.1 System parameters

Parameter	Value	Note
Payload mass (belt + holder + ball)	40 g	$m = 0.040 \text{ kg}$
Pulley radius	10.5 mm	$r_p = 0.0105 \text{ m}$
Belt incline angle	55°	Geometry constraint
Target peak velocity	0.5 m/s	F3 performance target
Acceleration time (t_a)	0.2 s	Trapezoidal profile
Cycle time	1.0 s	Accel / const / decel

2.2 Torque calculations

Quantity	Formula	Result	Unit
Linear acceleration	$a = v / t_a = 0.5 / 0.2$	2.5	m/s^2
Angular acceleration	$\alpha = a / r_p = 2.5 / 0.0105$	238.1	rad/s^2
Load torque (gravity + friction)	$T_{\text{load}} = m \cdot g \cdot \sin(55^\circ) \cdot r_p$	≈ 0.32	$\text{N} \cdot \text{m}$
Acceleration	$T_{\text{acc}} =$	≈ 0.001	$\text{N} \cdot \text{m}$

Quantity	Formula	Result	Unit
torque	$(m \cdot r_p^2) \cdot \alpha$		
Peak torque (T _{peak})	T _{load} + T _{acc} + T _{friction}	4.84	N·m
RMS torque (T _{rms})	$\sqrt{[(T_{peak}^2 \cdot t_a + T_{load}^2 \cdot t_c) / t_{cycle}]}$	≈ 3.16	N·m

2.3 Actuator selection

Required: rated torque ≥ 5 N·m (peak), continuous ≥ 3.5 N·m. The standard SG90 (0.018 N·m stall) is completely ruled out for this subsystem.

Recommended actuators:

Option	Stall torque	Speed	Notes
MG996R servo (geared)	≥ 9.4 kg·cm (≈ 0.92 N·m)	0.17 s/60°	Needs gear reduction stage
High-torque servo (DS3218)	≥ 20 kg·cm (≈ 1.96 N·m)	0.18 s/60°	Better fit with reduction
NEMA-17 stepper (high torque)	≥ 0.59 N·m holding	Up to 500 rpm	Best positional control

3. Shooting Mechanism

N20 DC gear motor — rotating arm

3.1 System parameters

Parameter	Value	Note
Ball mass	7.3 g	PLA, ~15% infill
Arm length	50 mm	L = 0.05 m
Required launch velocity	2.7 m/s	From projectile motion

Parameter	Value	Note
Horizontal target distance	30.76 cm	Design range
Vertical drop to basket	12 cm	Height differential
Launch angle	35°	Optimised trajectory
Acceleration time	0.2 s	Duty: 0.2 s on / 0.8 s off

3.2 Torque calculations

Governing projectile equation: $v = \sqrt{[g \cdot d^2 / (2 \cdot \cos^2(\theta) \cdot (d \cdot \tan(\theta) - h))]}$

Quantity	Formula	Result	Unit
Required angular velocity	$\omega = v / L = 2.7 / 0.05$	54 rad/s ($\approx$ 516 rpm)	rad/s
Conservative bound (2.06 m/s)	$\omega = 2.06 / 0.05$	41.2 rad/s ($\approx$ 394 rpm)	rad/s
Rotational inertia	$I = m \cdot L^2 = 0.0073 \times 0.05^2$	1.825×10^{-5}	kg·m ²
Angular acceleration	$\alpha = \Delta\omega / t_a = 41.2 / 0.2$	206	rad/s ²
Peak torque (τ_{peak})	$I \cdot \alpha + m \cdot g \cdot L$	0.0157	N·m
RMS torque (τ_{rms})	$\tau_{peak} \cdot \sqrt{t_a / t_{cycle}}$	≈ 0.0070	N·m

3.3 Actuator selection

Required: rated speed $\geq$ 400 rpm; stall torque $\geq$ 0.020 N·m; voltage 6–12 V.
 Selected: N20 DC Gear Motor, 12 V, $\sim$ 400 rpm, stall torque $\approx$ 0.025 N·m.
 Safety factor $\approx$ 1.27 $\times$.

Driver: L298N H-bridge module. Arduino pins 12, 13 (direction) + pin 6 (PWM speed). External 12 V supply.

Note: The safety margin of 1.27× is conservative — consider a higher-torque N20 variant if ball mass or arm length deviates from specification.

4. Sorting Mechanism

SG90 micro servo — rotary gate

4.1 System parameters

Parameter	Value	Note
Rotation angle per sort	45° (0.785 rad)	One sorting sweep
Motion time	0.2 s	Gate travel
Reset time	0.1 s	Return to neutral
Gate mass (approx)	5 g	3D-printed PLA disc
Gate radius (approx)	15 mm	Thin disc approx.
Sorting accuracy target	≥ 95%	≥ 19/20 balls correct

4.2 Torque calculations

Quantity	Formula	Result	Unit
Angular acceleration	$\alpha = 2\theta / t^2 = 2 \times 0.785 / 0.04$	39.25	rad/s ²
Gate inertia (thin disc)	$I = \frac{1}{2} \cdot m \cdot r^2 = \frac{1}{2} \times 0.005 \times 0.015^2$	5.625×10^{-7}	kg·m ²
Peak torque (τ_{peak})	$I \cdot \alpha + \tau_{\text{gravity}}$	≈ 0.00365	N·m
RMS torque	$\tau_{\text{peak}} \cdot \sqrt{0.2 / 0.04}$	≈ 0.00165	N·m

Quantity	Formula	Result	Unit
(τ_{rms})	1.0)		
SG90 stall torque (@ 4.8 V)	Datasheet rated	0.018	N·m
Safety factor	$0.018 / 0.00365$	$\approx 4.9\times$	—

4.3 Actuator selection

SG90 micro servo is well within rating with a $4.9\times$ safety factor. No upgrade required.

PWM signal: pin 0, sweep range 0° – 55° . Power: 5 V from Arduino rail.

5. Actuator Sizing Summary

Subsystem	T_{peak} (N·m)	T_{rms} (N·m)	Rated (N·m)	Safety factor	Selected actuator
Lifting	4.84	3.16	≥ 5.0	$\approx 1.03\times$	High-torque servo / NEMA-17
Shooting	0.0157	0.0070	≥ 0.020	$\approx 1.27\times$	N20 DC gear motor (12 V)
Sorting	0.00365	0.00165	0.018	$\approx 4.9\times$	SG90 micro servo

Key observations:

The lifting mechanism is the most demanding actuator. A standard SG90 or MG996R without gear reduction is insufficient. A high-torque servo (DS3218 class) with an appropriate reduction ratio, or a NEMA-17 stepper driven by an A4988/DRV8825 driver, is recommended.

The shooting motor safety margin (1.27×) is intentionally lean to keep rpm within the N20's operational range. Verify actual ball mass and arm length before finalising the motor variant.

The SG90 sorting servo has substantial headroom and is correctly specified. No change required.

Projectile Motion Calculations

1. Given Parameters

The following physical constraints are defined for the ball's trajectory:

Horizontal Distance (d): 30.76 cm (0.3076 m)

Vertical Distance (h): 12 cm (0.12 m)

Acceleration due to Gravity (g): 9.81 m/s²

2. Launch Configuration

A practical launch angle was selected to ensure a consistent curved trajectory to the target:

Launch Angle (θ): 35 degrees

3. Mathematical Model

The required initial velocity is determined using the projectile motion equation for a target at a specific height and distance:

$$v = \sqrt{ \left(\frac{g \cdot d^2}{2 \cdot \cos^2(\theta) \cdot (d \cdot \tan(\theta) - h)} \right) }$$

3.1. Substitution and Intermediate Steps

Trigonometric Values: $\tan(35^\circ) \approx 0.700$, $\cos(35^\circ) \approx 0.819$

Horizontal Displacement Component: $d \cdot \tan(\theta) = 0.3076 \times 0.700 = 0.215$

Relative Vertical Component: $d \cdot \tan(\theta) - h = 0.215 - 0.12 = 0.095$

4. Final Results and Conclusion

Based on the calculations, the required performance metrics for the shooting mechanism are:

Required Initial Velocity (v): ≈ 2.7 m/s

Target Launch Angle (θ): 35°

Conclusion: These parameters ensure the ball follows a stable projectile path to reach the target accurately.

5. Root Mean Square (RMS) Velocity Analysis

To assess the continuous power requirement for the launcher during repeated operation, an RMS calculation is applied. Assuming a 1.0-second

duty cycle where the acceleration to launch velocity occurs over 0.2 seconds:

Launch Velocity (vlaunch): 2.7 m/s

Acceleration Duration (tacc): 0.2 s

Idle/Reset Duration (tidle): 0.8 s

Total Cycle Time (ttotal): 1.0 s

Vrms Calculation:

$$V_{rms} = \sqrt{[(v_{launch}^2 \cdot t_{acc}) / t_{total}]}$$

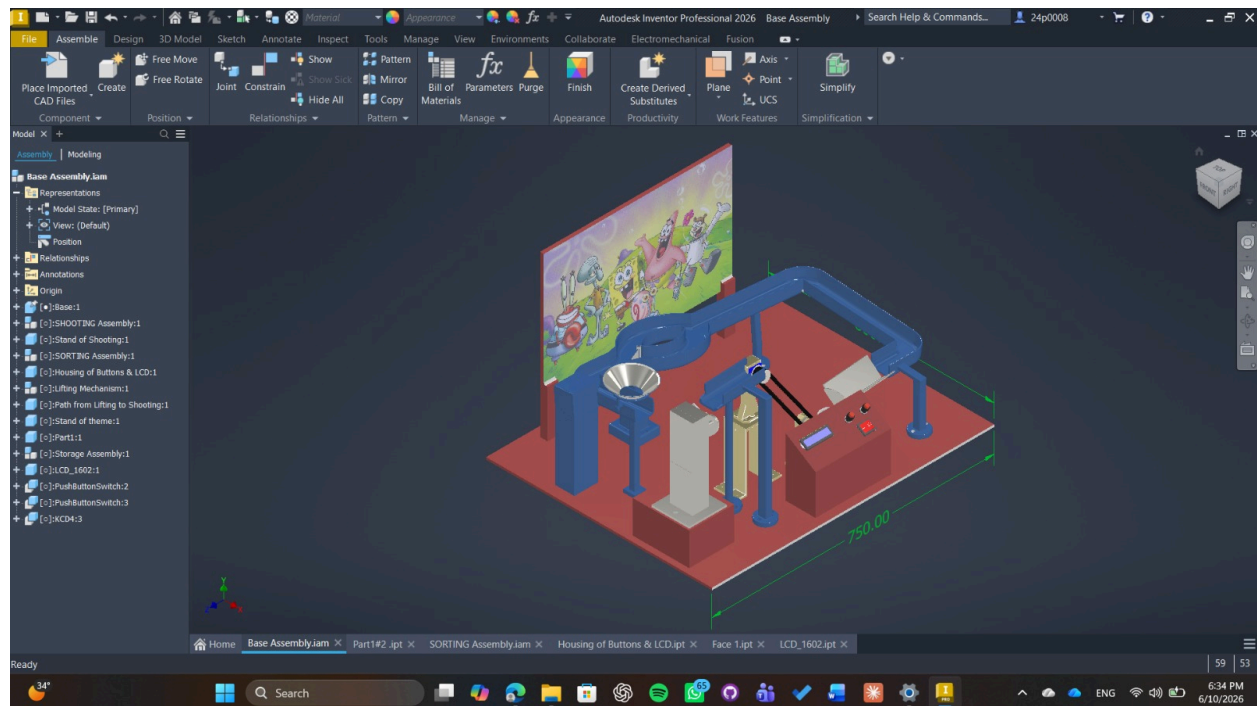
$$V_{rms} = \sqrt{[(2.7^2 \cdot 0.2) / 1.0]}$$

$$V_{rms} = \sqrt{[(7.29 \cdot 0.2) / 1.0]} = \sqrt{1.458}$$

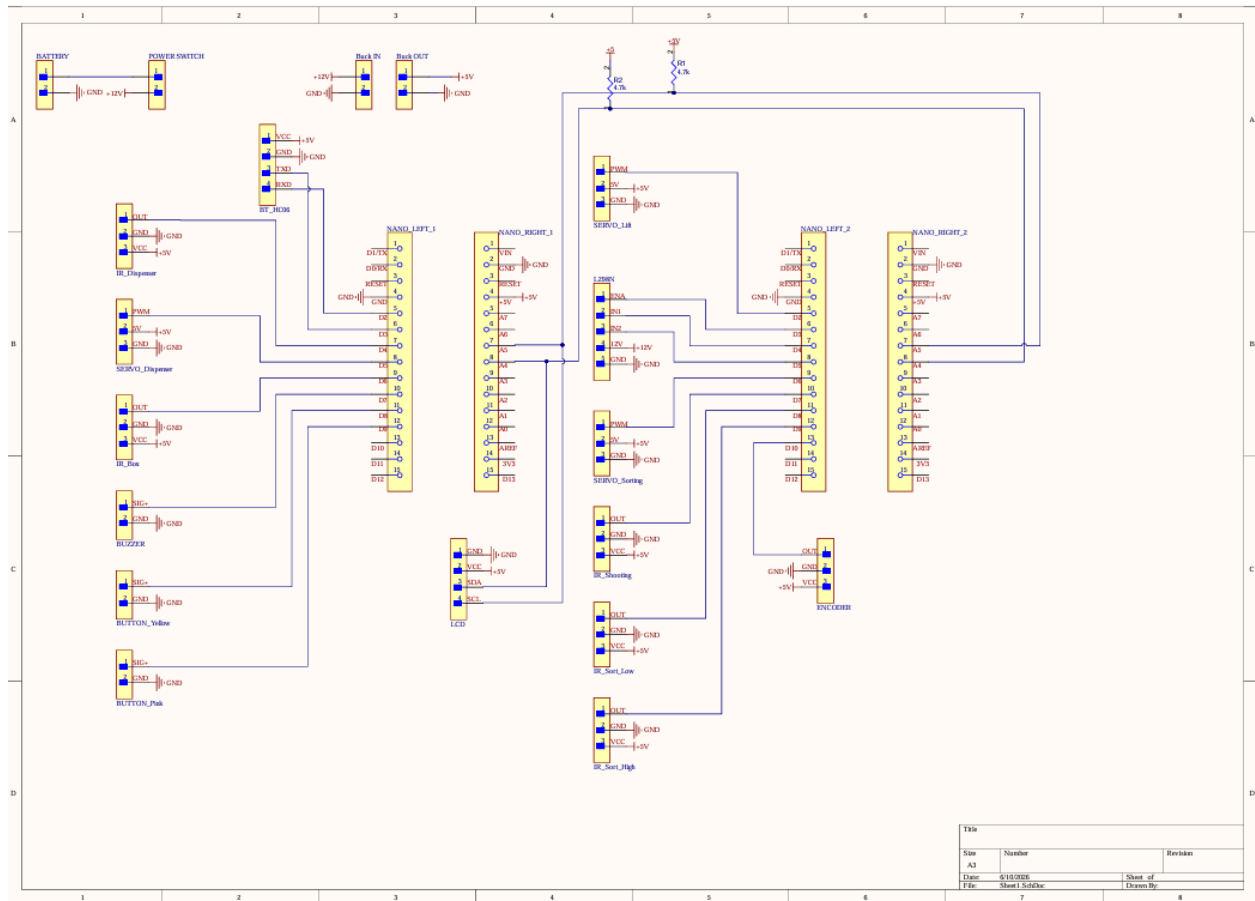
$$V_{rms} \approx 1.21 \text{ m/s}$$

Software designs:

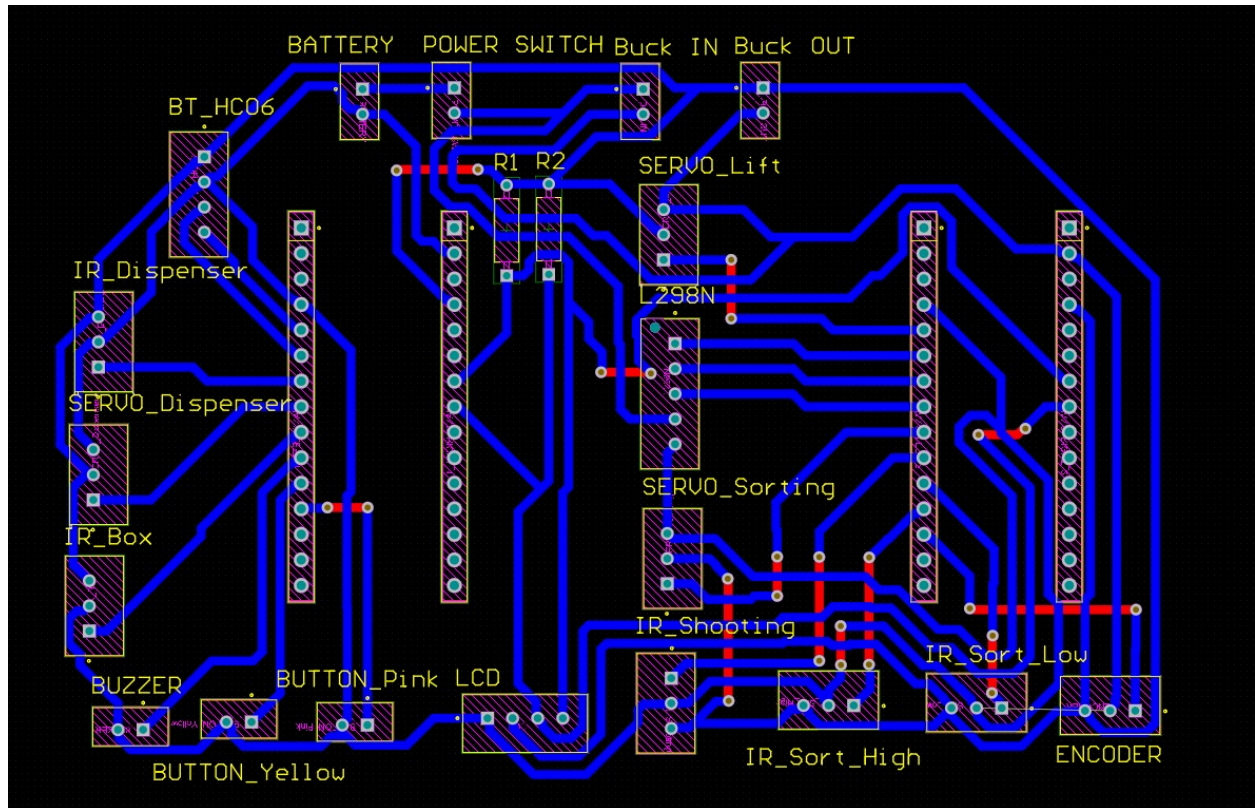
CAD assembly design:



Wiring diagrams:



PCB design:



Arduino programming:

Contribution Table:

Team Member	Primary Tasks	Weight (%)
Nada	Lifting mechanism design, GUI, system integration, LCD programming	20%
Ahmed	Shooting mechanism design, projectile calculations, and actuator sizing	20%

Team Member	Primary Tasks	Weight (%)
Rawaa	PCB design and wiring & Arduino programming	20%
Mohamed	Software and hardware fabrication and assembly, dispenser mechanism	20%
Farida	Sorting mechanism design& Arduino programming, PID control, supplementary CAD designs, CAD assembly	20%
Total Contribution:		100%

Conclusion:

This project successfully demonstrated the design and implementation of a fully automated SpongeBob-themed educational toy by integrating mechanical, electrical, electronic, and software subsystems into a single mechatronic system. The developed toy incorporates multiple mechanisms, including ball dispensing, lifting, shooting, and sorting, while providing an interactive learning experience for children through a color-recognition game.

The system utilized various sensors, servo motors, a DC motor with encoder feedback, an L298N motor driver, an LCD display, push buttons, a buzzer, and wireless Bluetooth communication to achieve reliable and autonomous operation. Closed-loop control was implemented through sensor feedback and PID-based motor control, ensuring accurate ball detection, controlled motion, and consistent system performance. The use of two Arduino Nano microcontrollers connected through the I²C protocol allowed the system to be modular, scalable, and easier to manage by distributing tasks between the Master and Slave controllers.

In addition to the mechanical functionality, the project incorporated a graphical user interface (GUI) that enabled remote monitoring and control of the toy through Bluetooth communication. The LCD display, buzzer, and scoring system enhanced the user experience by providing interactive feedback and encouraging children to participate in the educational activity. The color-identification game successfully combined entertainment with learning, demonstrating how mechatronic systems can be used to support educational objectives.

From an engineering perspective, the project provided practical experience in embedded systems programming, sensor integration, motor control, PCB design, wireless communication, power management, and system integration. It also highlighted the importance of combining hardware and software design to create reliable and user-friendly products.

Overall, the project achieved its intended objectives by creating a functional, automated, and interactive educational toy that showcases the principles of mechatronics engineering. The final system demonstrates how multiple subsystems can be integrated into a cohesive product that is both engaging and educational, while serving as a practical application of the concepts studied throughout the mechatronics course.

References:

Bolton, W. (2018). Mechatronics: Electronic control systems in mechanical and electrical engineering. Pearson Education.

Beer, F. P., Johnston, E. R., & Cornwell, P. J. (2019). Vector mechanics for engineers: Dynamics. McGraw-Hill Education.